A Hybrid-Parallel Decision Tree: MPI, OpenMP and CUDA Scaling of Histogram-Based CART

Jen-Chien Huang (A0353871) Bing Wen Lim Haoran Zhang

https://github.com/dannyhuang1111/dsa5208-distributed-decision-tree

1 Introduction

A decision tree is built greedily: starting from all the training rows at the root, each node chooses the one feature and threshold that best separates the two classes, splits its rows accordingly, and repeats on the two children until a stopping rule fires. This project parallelizes the *construction* of a binary classification tree and studies how different parallelization choices affect its scaling and efficiency.

The problem is a good parallelization target because it is *not* embarrassingly parallel. Although the training rows can be partitioned across processes trivially, choosing a split needs the class counts on both sides of every candidate threshold *summed over all rows*. With the rows spread across processes, no process can decide a split alone: each node forces a collective communication, and the children a node produces determine the work of the next level. That per-node global reduction, and the dependency between levels, are the source of the communication and load-balancing challenges we examine.

The report follows the project from the algorithm and its serial cost model (Sections 2–3), through a correctness validation (Section 4), to the three scaling studies: serial problem-size scaling (Section 5), pure-MPI strong and weak scaling to 256 cores (Section 6), and the MPI+X results for the MPI+OpenMP hybrid and the CUDA GPU arm (Section 7). Reproduction instructions are in `README.md`.

2 The algorithm and the computing environment

2.1 Histogram-based CART with the Gini criterion

For a node holding t samples of which p are positive, the Gini impurity is

$$G(p,t) = 1 - \left(\tfrac{p}{t}\right)^2 - \left(\tfrac{t-p}{t}\right)^2, \tag{1}$$

and a candidate split into a left child (p_L, t_L) and right child (p_R, t_R) is scored by the Gini gain

$$\Delta G = G(p,t) - \frac{t_L}{t}\, G(p_L, t_L) - \frac{t_R}{t}\, G(p_R, t_R). \tag{2}$$

The node adopts the (feature, threshold) with the largest ΔG, and recursion stops when the node reaches the maximum depth, holds at most `min-split` samples, is pure, or no split gives positive gain.

Rather than sorting each feature at every node, we use a **histogram** formulation. Each feature's values are placed into bins; when a feature has at most 256 distinct values the bins are its exact distinct values, so the candidate thresholds are identical to a sort-based search. A node is then summarised by, per bin, the pair [count, positive count]. Scanning those bins in order reproduces every candidate split from Equation 2. The histogram is the unit of work that the parallelization distributes, and because its entries are integers, summing them across processes is exact and order-independent — a property we rely on for both correctness and determinism (Sections 3–4).

Per node, the dominant cost is building the histogram, which is $O(n_{\text{node}} \times F)$ for n_{node} rows and F features. Summed over the tree, training cost grows with both the number of rows N and the number of features F; Section 5 measures both.

2.2 One source, four configurations

The algorithm is written once in `src/dt.cpp`; MPI and OpenMP are enabled by compile-time guards so the split-search logic is byte-identical in every build. This yields the four configurations the report compares:

- **serial** — no MPI, no OpenMP; the single-core baseline for speedup;
- **pure MPI** — the MPI build run with `OMP_NUM_THREADS=1`, so MPI is the only parallelism (Section 6);
- **MPI+OpenMP** — the same build with multiple threads per rank (Section 7);
- **CUDA** — a separate GPU implementation of the histogram hot path (Section 7).

2.3 Computing environment

Benchmarks were run on the NUS **Vanda** HPC cluster. The 256-core CPU scaling run recorded in `results/raw/env_20261110_224241_1537141.txt` (PBS job `1537141.stdct-mgmt-02`, head compute node `CN-064`) used four dual-socket **Intel Xeon Platinum 8452Y** nodes (36 cores per socket, 72 physical cores per node, one hardware thread per core, 64 cores allocated per node for 256 cores total), **OpenMPI 4.1.4**, and **GCC 12.2.0**; each benchmark point is the median of 3 repetitions after a warm-up tree. The GPU benchmarks (PBS jobs `1536934` and `1537453`) ran on a Vanda GPU node equipped with an **NVIDIA A40** (48 GB GDDR6, Ampere architecture) compiled with **CUDA 12.2**.

2.4 Dataset

We use the preprocessed Instacart reorder dataset `instacart_clean.csv`: 200 000 orders, 7 features (order number, day of week, hour of day, days since prior order, add-to-cart order, aisle id, department id) and a binary `reordered` label. The file is clean (no missing values) and every feature has at most 256 distinct values, so binning is exact. Plainly stated, the `instacart_clean.csv` committed in the repository is the schema-matched sample generated by `preprocess.py` (which plants higher reorder probabilities for `add_to_cart_order` ≤ 3 and `days_since_prior_order` $= 7$ when the raw multi-gigabyte Kaggle CSVs are not present locally).

Because 200 000 rows are far too few to occupy hundreds of cores — the per-node reduction would dominate — the scaling studies instead use a deterministic synthetic generator that reproduces the Instacart schema and value ranges. Row r is a pure function of (seed, r), so the dataset is identical for every process count and nothing is read from disk, letting us vary N and F freely.

The synthetic data is Instacart-*like* in shape, not sampled from the real file; real-data conclusions (Section 8) come from `instacart_clean.csv` itself.

3 Parallelization design

Data parallelism over rows, not over tree nodes. The rows are block-distributed across processes once, at load time, and never move again. For each node, every process builds the histogram of *its own* rows and a single `MPI_Allreduce` with integer sum produces the global histogram; every process then scans that identical histogram and independently reaches the identical split, so no split decision needs to be broadcast.

We deliberately did not parallelize by handing whole subtrees to different processes (task parallelism). Near the root there are only a handful of nodes but the most data, so subtree parallelism is scarcest exactly where the cost is greatest; sibling subtrees are unbalanced, so processes would idle; and assigning a subtree to a process would require reshuffling rows at every split. Data parallelism instead keeps every process busy on every node and attacks the cost where it lives — the histogram build, which Section 6 shows dominates the runtime.

Determinism as a design constraint. Because the reduction sums integers, its result does not depend on the number of processes or the order of summation, so the tree is bit-for-bit identical across all configurations. This both gives a strong correctness check (Section 4) and ruled out alternatives that would have broken it, such as reducing floating-point partial gains or assigning subtrees to processes.

Two communication patterns (tree growth). The same tree can be grown two ways, trading off message count against message size. *Node-wise* growth is depth-first with one reduction per internal node (many small messages, latency-bound); *level-wise* growth is breadth-first and reduces the stacked histograms of all nodes at one depth in a single collective (far fewer, larger messages). At depth 6 on $N = 32\,000\,000, F = 28$ (Table 1 and Figure 1), node-wise growth issues 64 `MPI_Allreduce` calls whereas level-wise issues only 7, while both exchange the exact same total payload (0.6955 MB). Eliminating 57 collective round-trips reduces training time at 256 cores from 0.1733 s to 0.0634 s for 64×4 (2.73× faster) and from 0.1075 s to 0.0652 s for 256×1 (1.65× faster).

Two OpenMP strategies (X = OpenMP). Within a rank, threads can either split the *rows* (each thread fills a private histogram, then a parallel reduction combines them) or split the *features* (threads share one histogram, with parallelism capped by F). As Table 1 confirms, splitting rows (`sample`) beats splitting features (`feature`) across every configuration: at 64×4 with level-wise growth, `sample` runs in 0.0634 s versus 0.1530 s for `feature` (2.41× faster), because row parallelism scales with N/p rather than being bottlenecked by $F = 28$.

Table 1: Design-choice comparison at 256 cores ($N = 32\,000\,000, F = 28$, depth 6; from `design_20261110_224241_1537141.csv`).

Ranks	Threads	Growth	OpenMP	`Allreduce` calls	Median time (s)
64	4	level	sample	7	**0.0634**
64	4	level	feature	7	0.1530
64	4	node	sample	64	0.1733
64	4	node	feature	64	0.2740
256	1	level	sample	7	**0.0652**
256	1	level	feature	7	0.0806
256	1	node	sample	64	0.1075
256	1	node	feature	64	0.1161

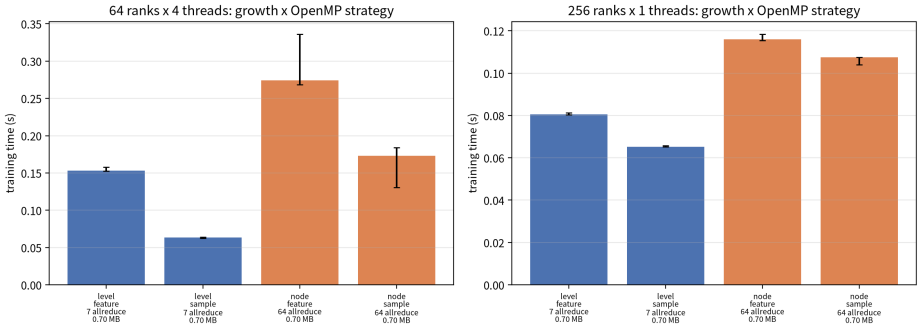


Figure 1: Level-wise vs. node-wise growth and sample vs. feature OpenMP strategies on Vanda (256 cores, $N = 32\,000\,000, F = 28$).

4 Validating correctness

Two independent checks establish that the parallel builds compute the right tree.

First, **cross-configuration equivalence** exploits the determinism above. The script `scripts/check_equivalenc` hashes the entire tree (structure, split features, thresholds and leaf predictions) with FNV-1a and requires every configuration — process counts $\{1, 2, 3, 4, 7\}$, thread counts $\{1, 2, 3\}$, both growth modes, both OpenMP strategies, both the synthetic and CSV input paths, on a prime number of rows ($N = 100\,003$) so the row blocks are uneven — to reproduce the serial reference hash (`0bfc2e3832b96e3f`) exactly, reporting:

```
64 / 64 configurations identical to the reference
```

In addition, `analysis/plot_results.py` verifies tree hashes across all 969 benchmark runs in `results/raw/`: every (N, F, depth) dataset produces exactly 1 distinct tree hash across all 33 parallel configurations (e.g. hash `50e4598e0ff9ac07` across all 222 runs at $N = 32\,000\,000, F = 28$).

Second, an **external reference**: `scripts/compare_sklearn.py` rebuilds the tree dumped by `dt --dump-tree` and compares it node by node against scikit-learn's `DecisionTreeClassifier` on `instacart_clean.csv`. Both build a 125-node tree with identical training accuracy (68.5035 %) and agree on 99.9850 % of row predictions (123 of 125 nodes match directly, with one exact-tie

leaf accepted and a single bottom-level tie at `rootRLRRR` where $f_4 \leq 5$ and $f_5 \leq 4.5$ yield equal impurity gain).

This equivalence was also what motivated the current design. An earlier MPI implementation combined rank-local split gains with `MPI_MAXLOC`, which is not the global Gini gain; its trees drifted with the process count (training accuracy 68.46 % at one rank versus 68.42 % at eight on the same 200 000 rows), while the root split stayed correct and so hid the bug from shallow checks. The histogram-reduction design removes this drift completely.

5 Serial scaling with problem size

The serial build establishes both the speedup baseline and the cost model. We vary the two dimensions of problem size independently: the number of rows $N \in \{1, 2, 4, 8, 16, 32\} \times 10^6$ at fixed $F = 28$, and the number of features $F \in \{7, 14, 28, 56, 112, 224\}$ at fixed $N = 8\,000\,000$. Because per-node work is $O(n_{\text{node}} \times F)$, both should give near-linear runtime growth.

Table 2: Serial runtime sweep varying N at $F = 28$ (left) and varying F at $N = 8\,000\,000$ (right) on one Xeon Platinum 8452Y core (`serial_scan_20261110_224241_1537141.csv`).

Rows N ($F = 28$)	Median T_1 (s)	Features F ($N = 8$M)	Median T_1 (s)
1 000 000	0.2392	7	0.4669
2 000 000	0.2998	14	0.7027
4 000 000	0.6080	28	1.2012
8 000 000	1.2012	56	2.3235
16 000 000	2.5358	112	4.7411
32 000 000	4.8806	224	10.5904

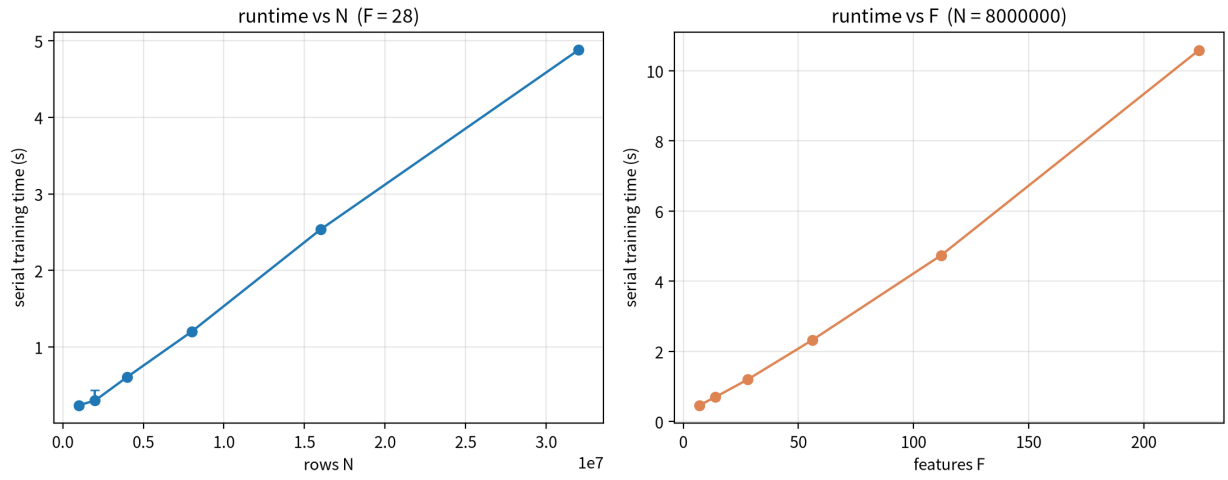


Figure 2: Serial training time vs. rows N at $F = 28$ (left) and vs. features F at $N = 8\,000\,000$ (right) on Vanda.

Table 2 and Figure 2 confirm the $O(N \times F)$ cost model along both axes. Doubling N from 4M to 32M at $F = 28$ scales runtime linearly with a constant slope of 0.152 µs per row (32M/4M = 8× data gives 4.881/0.608 = 8.03× runtime). Likewise, doubling F from 14 to 224 at $N = 8M$ increases runtime from 0.703 s to 10.590 s (16× features gives 15.07× runtime, slope ≈ 0.046 s per feature).

6 Pure-MPI strong and weak scaling

This section uses the pure-MPI configuration (one rank per core, `OMP_NUM_THREADS=1`), alongside the 4-thread-per-rank hybrid curve for reference. Speedup is $S_p = T_1/T_p$ against the serial baseline on the same data, and parallel efficiency is $E_p = S_p/p$ for p cores; T_p is the maximum over ranks of the barrier-to-barrier training time.

6.1 Strong scaling

Here the problem is fixed at $N = 32\,000\,000$ rows and the core count grows from 1 to 256 cores across four Vanda nodes ($F \in \{7, 14, 28, 56\}$; we focus on $F = 28$ in Table 3 and Figure 3).

Table 3: Strong scaling to 256 cores on Vanda ($N = 32\,000\,000, F = 28$, depth 6, $T_{\text{serial}} = 5.169$ s; from `strong_scaling_F28.csv`).

Cores p	Pure MPI T_p (s)	Speedup S_p	Efficiency E_p	Hybrid ($t = 4$) T_p (s)	Hybrid S_p
1	5.353	0.97×	96.6%	—	—
2	2.972	1.74×	87.0%	—	—
4	1.412	3.66×	91.5%	1.405	3.68×
8	0.945	5.47×	68.4%	0.902	5.73×
16	0.420	12.31×	76.9%	0.523	9.89×
32	0.293	17.67×	55.2%	0.349	14.80×
64	0.202	25.64×	40.1%	0.187	27.61×
128	0.102	50.76×	39.7%	0.117	44.32×
256	**0.0684**	**75.54**×	**29.5%**	**0.0717**	**72.10**×

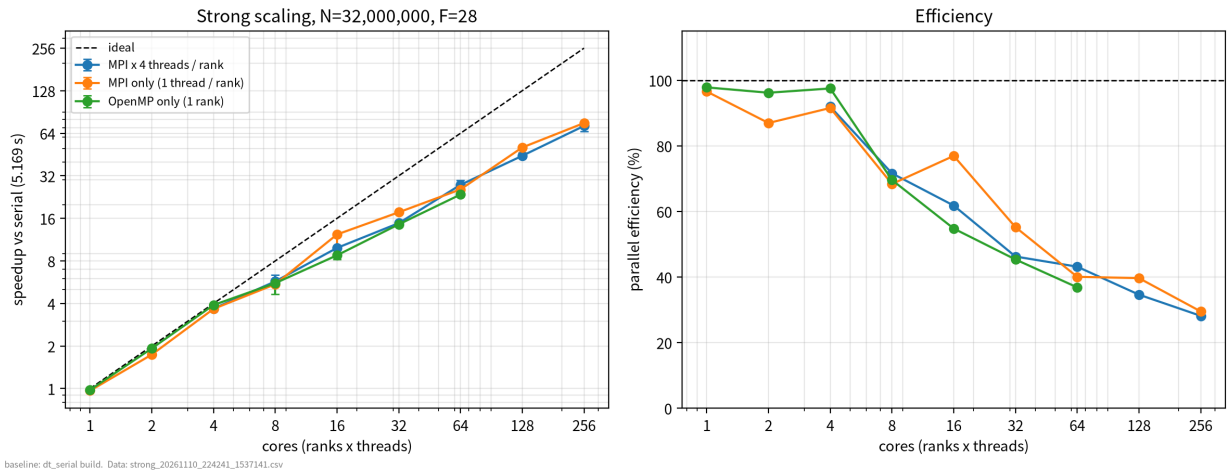


Figure 3: Strong scaling speedup (left) and parallel efficiency (right) to 256 cores on Vanda ($N = 32\,000\,000, F = 28$).

Pure MPI reduces training time from 5.169 s to 0.0684 s at 256 cores (75.54× speedup, 29.5 % efficiency), while the 64×4 hybrid reaches 0.0717 s (72.10×). Across the feature sweep at 256 cores, pure-MPI speedup increases with F: 71.33× at $F = 7$, 56.06× at $F = 14$, 75.54× at $F = 28$, and 95.34× (37.2 % efficiency, 9.885 s down to 0.1037 s) at $F = 56$. Efficiency stays above 75 % up to 16 cores, steps down at 32–64 cores as a single node's memory channels saturate, and tapers to 29.5 % at 256 cores once per-rank histogram work shrinks to $\sim$ 40 ms and multi-node `MPI_Allreduce` latency becomes a larger fraction of total time.

6.2 Weak scaling

Here the work per core is fixed at 1 000 000 rows per core and both N and p grow together from $N = 1\,000\,000$ ($p = 1$) to $N = 256\,000\,000$ ($p = 256$).

Table 4: Weak scaling to 256 cores / 256M rows at 1M rows per core ($F = 28$, depth 6; from `weak_scaling.csv`).

Cores p	Total rows N	Pure MPI T_p (s)	Weak E_p	Hybrid ($t = 4$) T_p (s)	Hybrid E_p
1	1 000 000	0.2563	100.0%	—	—
2	2 000 000	0.2546	100.7%	—	—
4	4 000 000	0.2720	94.2%	0.2523	100.0%
8	8 000 000	0.3159	81.1%	0.2662	94.8%
16	16 000 000	0.3410	75.2%	0.2896	87.1%
32	32 000 000	0.3532	72.6%	0.4003	63.0%
64	64 000 000	0.4296	59.7%	0.4013	62.9%
128	128 000 000	0.4928	52.0%	0.4235	59.6%
256	256 000 000	**0.4863**	**52.7%**	**0.5884**	**42.9%**

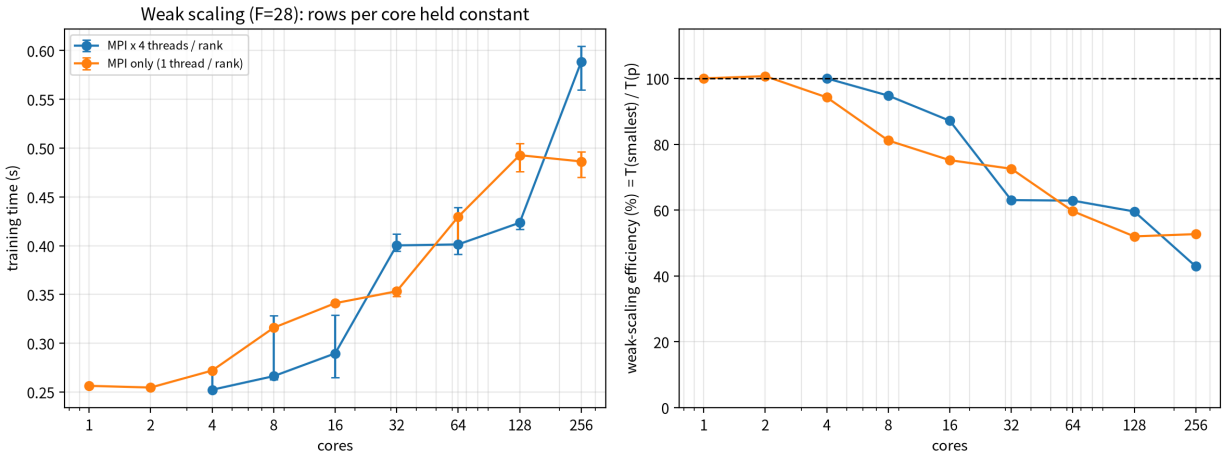


Figure 4: Weak scaling runtime (left) and efficiency (right) at 1 000 000 rows/core up to 256 cores (256M rows, $F = 28$) on Vanda.

Table 4 and Figure 4 show that scaling problem size 256-fold (from 1M to 256M rows across four nodes) increases pure-MPI wall time by less than 1.9× (0.256 s to 0.486 s), sustaining 52.7 % **weak scaling efficiency** at 256 cores (57.2 % at $F = 7$, 64.3 % at $F = 14$, and 54.4 % at $F = 56$). Because per-core compute work is constant, the gradual drop from 8 to 64 cores reflects shared memory-bandwidth contention within a 64-core node, while the step beyond 64 cores reflects inter-node collective communication.

6.3 The impact of communication

With `--profile`, a barrier before each reduction separates time spent *waiting* for the slowest rank (load imbalance) from time spent in the reduction itself (transfer). Figure 5 and Table 5 decompose the profiled $F = 56$ strong-scaling runs into histogram build (t_{hist}), split scan (t_{scan}), row partition (t_{part}), barrier wait (t_{wait}) and `MPI_Allreduce` transfer ($t_{\text{allreduce}}$).

Table 5: Profiled runtime breakdown for pure MPI ($N = 32\,000\,000, F = 56$, depth 6; from `strong_breakdown.csv`).

Cores p	t_{total} (s)	t_{hist} (s)	t_{part} (s)	t_{wait} (s)	$t_{\text{allreduce}}$ (s)	Comm + wait share
1	10.570	9.092	1.466	0.0000	0.0000	0.0%
8	1.468	1.244	0.193	0.0258	0.0029	2.0%
32	0.556	0.459	0.056	0.0319	0.0077	7.1%
64	0.293	0.219	0.029	0.0176	0.0256	14.7%
128	0.182	0.123	0.015	0.0124	0.0300	23.3%
256	0.098	0.060	0.008	0.0138	0.0148	**29.2%**

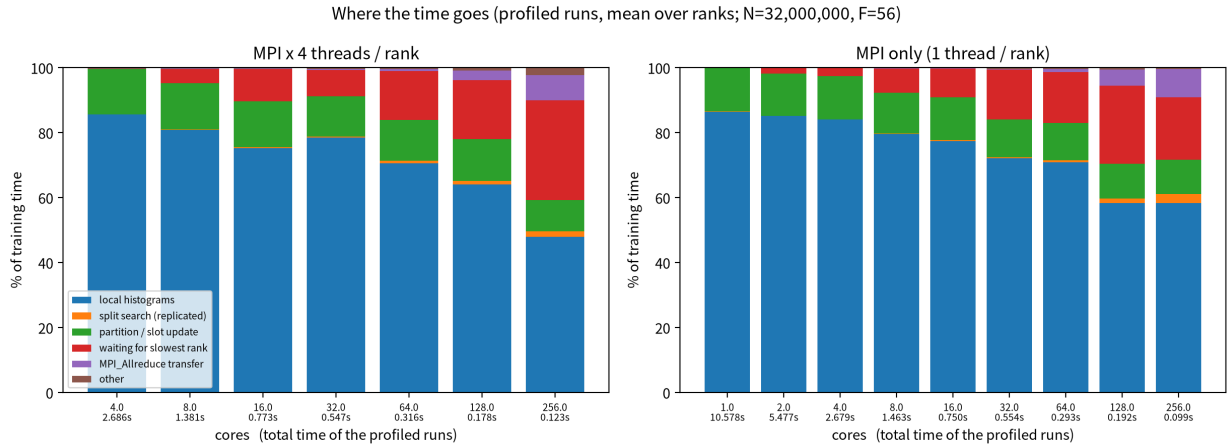


Figure 5: Stacked phase breakdown vs. core count on Vanda ($N = 32\,000\,000, F = 56$): t_{hist} and t_{part} shrink as $O(1/p)$ while $t_{\text{wait}} + t_{\text{allreduce}}$ grows to 29.2 % of runtime at 256 cores.

At $p = 1$, histogram construction (86.0 %) and partitioning (13.9 %) account for 99.9 % of the runtime. Both parallel phases shrink nearly ideally with p: from 1 to 256 cores, t_{hist} drops 152× (9.092 s → 0.060 s) and t_{part} drops 188× (1.466 s → 0.0078 s). In contrast, synchronization wait and `MPI_Allreduce` transfer grow from 2.0 % at 8 cores to 14.7 % at 64 cores and 29.2 % (28.6 ms out of 98.0 ms) at 256 cores across four nodes, explaining why strong-scaling efficiency settles around 30 %–37 % at 256 cores.

7 MPI+X: OpenMP hybrid and the GPU

7.1 MPI + OpenMP

Adding OpenMP threads within each rank reduces the number of ranks sharing the reduction while keeping the cores busy. At a fixed total budget of 256 cores ($N = 32\,000\,000, F = 28$), we sweep the ranks × threads split from 256 × 1 to 4 × 64 (Table 6 and Figure 6).

Table 6: Runtime at fixed 256 cores across ranks × threads splits ($N = 32\,000\,000, F = 28$; from `hybrid_fixed.csv`).

Ranks × Threads	Total cores	Median T (s)	Min T (s)	Max T (s)
256 × 1	256	0.0686	0.0681	0.0687
128 × 2	256	0.0689	0.0679	0.0718
64 × 4	256	0.0693	0.0678	0.0694
32 × 8	256	0.0649	0.0648	0.0671
16 × 16	256	0.0721	0.0719	0.0737
8 × 32	256	0.0528	0.0526	0.0591
4 × 64	256	**0.0479**	**0.0478**	0.0574

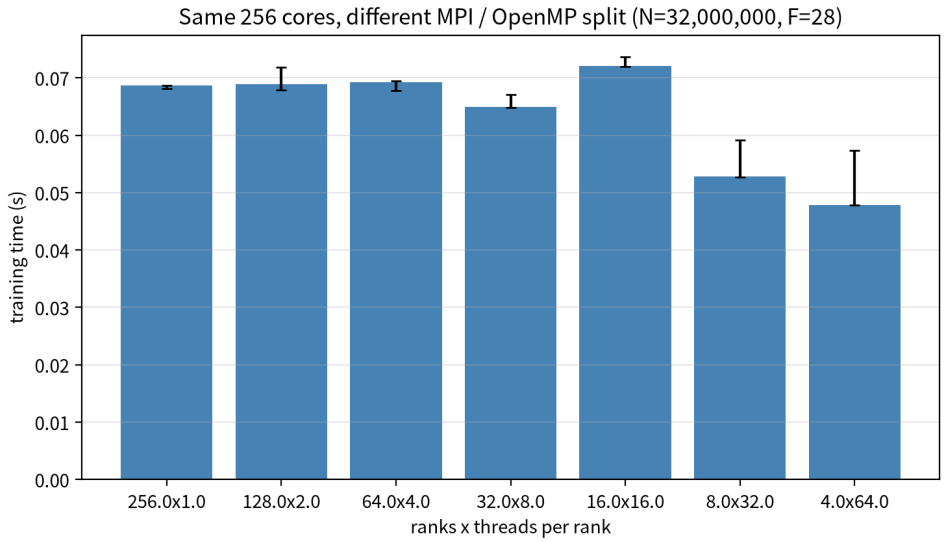


Figure 6: Fixed 256-core runtime across MPI × OpenMP splits on Vanda ($N = 32\,000\,000, F = 28$).

Across 1 to 8 threads per rank (256×1 through 32×8), median runtime is nearly flat (0.0649 s–0.0693 s), while one rank per socket (8×32, 0.0528 s) and one rank per node (4×64, 0.0479 s, or 107.9× faster than serial) achieve the fastest times at 256 cores. With 4×64, the inter-node `MPI_Allreduce` involves only 4 processes instead of 256, slashing collective communication overhead while OpenMP's thread-local histogram reduction aggregates counts in shared L3 cache within each node. Conversely, in the single-node strong-scaling sweep (`strong_thr`), pure OpenMP on one node reaches 0.219 s (23.6×) at 64 threads versus 0.202 s (25.6×) for pure MPI, because thread-private histogram reduction across dual sockets incurs NUMA traffic when there is no inter-node MPI latency to save.

7.2 CUDA on a GPU

Because the chosen X is OpenMP, we additionally implement the histogram hot path on a GPU in CUDA (`src/tree_cuda.cu`) and benchmark it on a Vanda **NVIDIA A40** GPU. The kernel builds the exact same $F \times 256 \times 2$ integer [count, positive count] histogram as the CPU, using thread-block shared-memory privatization followed by global atomic accumulation and a parallel split-scan kernel, so the split chosen matches the CPU result identically.

Table 7: NVIDIA A40 GPU vs. CPU histogram and root-split benchmark on Vanda ($F = 28$; from `gpu_20261110_214122_1536934.csv` and `gpu_20261110_235027_1537453.csv`).

Dataset size	CPU 1-core hist	A40 H2D transfer	A40 kernel + scan	A40 total	Compute / To
$N = 2\,000\,000, F = 28$	0.3778 s	0.005 42 s	0.005 86 s	0.011 28 s	**64.44×**
$N = 32\,000\,000, F = 28$	0.6041 s	0.085 57 s	0.093 46 s	0.179 05 s	**6.46**

Table 7 reports both benchmark sizes:

- At $N = 2\,000\,000, F = 28$, the NVIDIA A40 builds the histogram and finds the root split in 5.86 ms of kernel time (11.28 ms including the 5.42 ms PCIe host-to-device copy), achieving a

64.44× **compute speedup** and 40.61× **end-to-end speedup** over one CPU core (0.3778 s histogram, 0.4580 s total). Both select feature 4, threshold 3.5, with Gini gain 0.016 713.

- At $N = 32\,000\,000, F = 28$ (1380 non-empty feature bins, 896 MB of uint8 feature data plus labels), the A40 kernel runs in 93.46 ms and the PCIe Gen4 host-to-device transfer takes 85.57 ms (10.47 GB/s), for an end-to-end GPU time of 0.1791 s versus 0.6041 s CPU histogram time (0.7390 s total depth-1 time) on one core — a 6.46× kernel speedup (4.13× including PCIe transfer), and matching a 4-core OpenMP node on the root pass. Both CPU and GPU choose the exact same split: feature 4, threshold 3.5, Gini gain 0.016 735.

8 Discussion and conclusions

How the problem was parallelized. Our implementation distributes rows statically across MPI ranks and represents each candidate split via exact integer histograms. Because histogram entries are additive integers, a single collective `MPI_Allreduce` per tree level suffices to give every rank the global class counts, guaranteeing bit-for-bit identical trees across serial, MPI, and OpenMP runs. Growing the tree level-wise batches all active nodes at a given depth into one reduction (7 collectives instead of 64 at depth 6, giving a 2.73× speedup at 256 cores), while row-parallel OpenMP threads within each rank reduce MPI communicator size and network contention. On the GPU, shared-memory histogram privatization yields up to 64.4× kernel speedup over a single CPU core. As the profiled breakdown in Section 6 demonstrates, local histogram construction scales as $O(N/p)$ and shrinks by 152× at 256 cores, leaving inter-node `MPI_Allreduce` latency and synchronization wait (29.2 % of runtime at 256 cores) as the primary scaling limit.

Conclusions about the data. On `instacart_clean.csv` ($N = 200\,000, F = 7$) the serial tree reaches 68.50 % training accuracy at depth 6, against a majority-class baseline of 66.17 % (67 669 of 200 000 rows, or 33.83 %, are reorders), a modest but consistent improvement that matches scikit-learn to 99.985 % prediction agreement. Inspecting `--dump-tree` shows that the root split is `add_to_cart_order` ≤ 3.5 (feature 4), immediately separating staple items placed into the cart first from later additions, while the next levels split on `days_since_prior_order` around 6.5–9.5 days (feature 3, capturing weekly replenishment cycles), `order_number` (feature 0), and `aisle_id` (feature 5). Finally, Figure 7 shows why scaling studies require synthetic data: on 200 000 rows, serial training takes only 24.1 ms, reaches a minimum of 1.69 ms at 32 cores (14.25× speedup), and then slows down at 64–256 cores (5.61 ms at 256 cores) because each rank holds only 781 rows and inter-node `MPI_Allreduce` latency completely dominates computation.

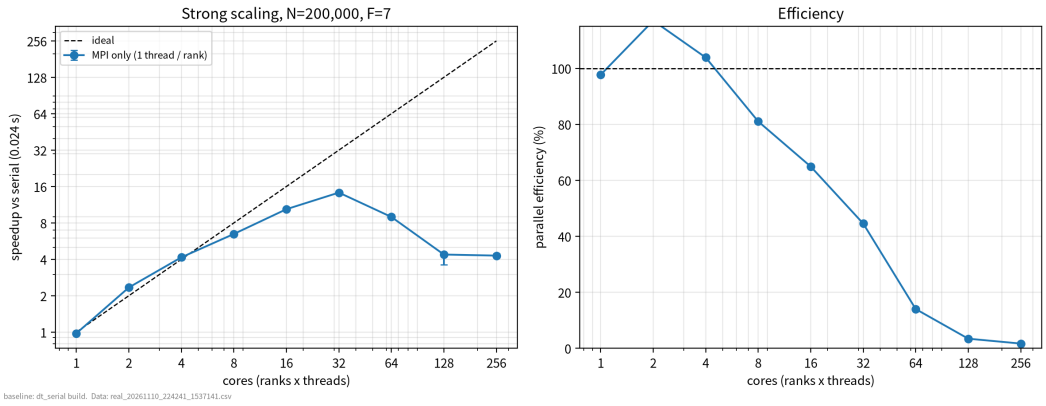


Figure 7: Strong scaling on the 200 000-row `instacart_clean.csv` dataset on Vanda: runtime bottoms out at 1.69 ms (32 cores, 14.25× speedup) before collective latency dominates.

9 AI usage

Anthropic's Claude (via the Claude Code CLI) and Google's Gemini (via Antigravity) assisted with code review, with diagnosing the earlier MPI split-search correctness issue, with PBS batch script troubleshooting, and with drafting and structuring this report. All design decisions, the choice of algorithm and parallelization strategy, and the final analysis were the authors'. All AI-assisted code was reviewed, compiled and run by the authors, and every reported number comes from an actual run on the NUS Vanda HPC cluster.